

Dad-A-Base

Architecture Overview

Version 1.1 · 2026-04-06

[lluppesms/dadabase.demo](#)

01

Executive Summary

1. Executive Summary

.NET 10 Blazor Server web applications with MudBlazor UI

Azure-hosted serverless APIs via Azure Functions (Flex Consumption)

Model Context Protocol (MCP) servers for AI agent integration

Dual data-source architecture (JSON fallback + Azure SQL)

Full Infrastructure-as-Code using Azure Bicep

Multi-platform CI/CD (GitHub Actions + Azure DevOps)

AI-powered features using Azure OpenAI and Microsoft Agent Framework

Playwright end-to-end testing and MSTest unit testing

02

Repository Structure

2. Repository Structure

```
dadabase.demo/  
├── src/  
│   ├── web/  
│   │   ├── Website/  
│   │   ├── Data/  
│   │   └── Tests/  
│   ├── function/  
│   │   ├── Function/  
│   │   ├── DataLayer/  
│   │   ├── Entities/  
│   │   └── Tests/  
│   ├── mcp/  
│   │   ├── DadJokeMCP/  
│   │   └── DadJokeMCPSSSE/  
│   ├── console/  
│   ├── analyzer/  
│   └── sql.database/  
├── infra/  
│   ├── Bicep/  
│   └── azd-main.bicep  
├── .github/  
│   ├── workflows/  
│   └── instructions/  
├── .azuredevops/  
│   └── policies/  
├── playwright/  
├── Docs/  
│   ├── Application-Architecture.md  
│   ├── Application-Architecture.pdf  
│   ├── Application-Architecture.pptx  
│   └── Export/  
├── TestHarness/  
└── azure.yaml
```

Application source code
Blazor Web App + Data layer
DadABase.Web (.NET 10 Blazor Server)
DadABase.Data (shared data library)
Unit tests (MSTest)
Azure Function (serverless API)
DadABase.Function (.NET 10 Isolated Worker)
Function-specific data layer
Function-specific entity models
Function unit tests
MCP Servers
Stdio-based MCP server
SSE-transport MCP server
Console application
Data analysis tool
SQL Database project (DACPAC)
Bicep IaC templates
Azure Developer CLI entry point
GitHub Actions CI/CD workflows
Copilot instructions and skills
Azure DevOps policy configuration
Playwright E2E test suites
Project documentation
Architecture documentation
Architecture document (PDF export)
Architecture presentation deck
Generated export artifacts
HTTP test files
Azure Developer CLI configuration

Monorepo: [src/](#) [infra/](#) [.github/](#) [playwright/](#) [Docs/](#)

03

Application Architecture

3. Application Architecture



3. Application Architecture

JokeJsonRepository — reads from Data/Jokes.json

JokeSQLRepository — uses EF Core with DadABaseDbContext

04

Source Code Projects

4. Source Code Projects

Property	Value
Path	src/web/Website/
Framework	.NET 10 (net10.0)
Project type	Microsoft.NET.Sdk.Web (Blazor Server)
UI library	MudBlazor 9.2.0
Solution	dadabase.net10.web.sln

4. Source Code Projects

Package	Version	Purpose
MudBlazor	9.2.0	Component library (UI)
Azure.AI.OpenAI	2.8.0-beta.1	Azure OpenAI SDK
Microsoft.Agents.AI.OpenAI	1.0.0-preview	Microsoft Agent Framework
Azure.Identity	1.20.0	Managed Identity / DefaultAzureCredential
Azure.Monitor.OpenTelemetry.AspNetCore	1.4.0	App Insights / OpenTelemetry
Azure.Storage.Blobs	12.27.0	Blob storage for images
Microsoft.EntityFrameworkCore.SqlServer	10.0.5	EF Core SQL provider
Microsoft.Identity.Web	4.7.0	Entra ID / OIDC authentication
Swashbuckle.AspNetCore	10.1.7	Swagger/OpenAPI
Blazored.LocalStorage	4.5.0	Browser local storage

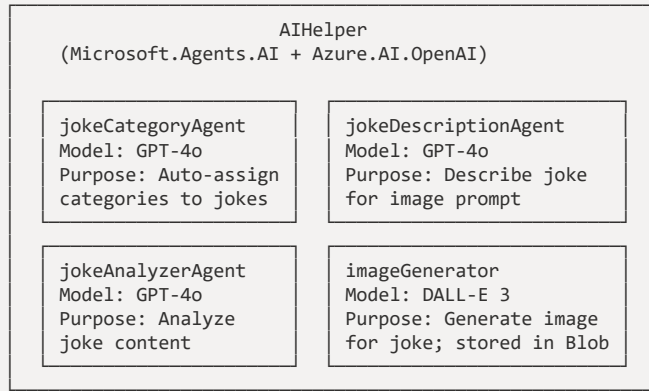
4. Source Code Projects

Page	Description
/ (Index)	Home page — displays a random joke with AI image generation
/Random	Random joke viewer
/Search	Joke search by text or category
/JokeDetail/{id}	Single joke detail view
/JokeEditor	Create / edit jokes with AI-powered category assignment
/Export	Export joke data
/Admin	Administration page
/About	About page

05

AI/GenAI Integration

5. AI / GenAI Integration



5. AI / GenAI Integration

AI Capabilities

Capability	Model	Description
Category classification	GPT-4o	Auto-assigns up to 2 categories to a new joke
Joke description	GPT-4o	Generates a scene description for DALL-E image prompting
Joke analysis	GPT-4o	General joke content analysis
Image generation	DALL-E 3	Creates an AI illustration per joke; saved to Azure Blob Storage

06

Infrastructure as Code (Bicep)

6. Infrastructure as Code (Bicep)

```
infra/Bicep/
├── main.bicep           Entry point – orchestrates all modules
├── main.bicepparam      Default parameter values
├── resourcenames.bicep  Centralised resource naming convention
├── modules/
│   ├── webapp/         Azure App Service plan + web app
│   ├── container/      Azure Container Apps + Container Registry
│   ├── function/       Azure Functions (Flex Consumption)
│   ├── functions/      Additional function app configuration
│   ├── database/       Azure SQL Server + database
│   ├── monitor/        Application Insights + Log Analytics
│   ├── security/       Azure Key Vault
│   ├── storage/        Azure Storage Accounts
│   └── iam/            Managed Identity + RBAC role assignments
└── data/
    ├── resourceAbbreviations.json  Resource type abbreviation lookup
    └── roleDefinitions.json        RBAC role definition IDs
```

deploymentType parameter: webapp | containerapp | functionapp | all

6. Infrastructure as Code

Key Azure Resources

Resource	Bicep Module	Notes
Azure App Service	modules/webapp/	Linux or Windows, configurable SKU (default: B1)
Azure Container Apps	modules/container/	With Container Registry (Basic/Standard/Premium)
Azure Functions	modules/function/	Flex Consumption plan, .NET 10 Isolated
Azure SQL Database	modules/database/	Default: GeneralPurpose GP_S_Gen5; configurable
Azure Key Vault	modules/security/	Secrets management; supports admin user access
Application Insights	modules/monitor/	OpenTelemetry exporter connected
Azure Storage	modules/storage/	Used by Function + web app (images)
Managed Identity	modules/iam/	User-assigned + system-assigned; role assignments

07

CI/CD Pipelines

7. CI/CD Pipelines — GitHub Actions

Workflow	File	Purpose
Deploy Infrastructure	1-deploy-bicep.yml	Provisions all Azure resources via Bicep
Build & Deploy Web (App Service)	2.1-bicep-build-deploy-webapp.yml	Build, test, and deploy Blazor app to App Service
Build & Deploy Web (Container App)	2.2-bicep-build-deploy-containerapp.yml	Build Docker image, push to ACR, deploy to Container Apps
Build & Deploy Function	3-bicep-build-deploy-function.yml	Build and deploy Azure Function
Deploy DACPAC	4-build-deploy-dacpac.yml	Build SQL project and deploy DACPAC to Azure SQL
Run SQL Script	5-run-sql-script.yml	Execute ad-hoc SQL scripts
PR Scan + Build	6-pr-scan-build.yml	Pull request validation gate
Code Scanning	7-scan-code.yml	Security scanning (CodeQL / GHAS)
Smoke Tests	8-smoke-test-webapp.yml	Post-deployment Playwright validation
AZD Deploy	azure-dev.yml	Azure Developer CLI full-stack deploy

7. CI/CD Pipelines — Azure DevOps

Template	Purpose
template-bicep-deploy.yml	Shared Bicep deploy step
template-webapp-build.yml	Shared .NET build + test step
template-webapp-deploy.yml	Shared App Service deploy step
template-containerapp-build.yml	Docker build + ACR push
template-containerapp-deploy.yml	Container Apps deploy
template-function-build.yml	Function build step
template-function-deploy.yml	Function deploy step
template-dacpac-build.yml	SQL project build
template-dacpac-deploy.yml	DACPAC deploy step
template-scan-code.yml	Code scanning step

08

Testing

8. Testing

Project	Location	Framework	Notes
Web unit tests	src/web/Tests/	MSTest + Coverlet	API controllers, data layer
Function unit tests	src/function/Tests/	MSTest	Trigger and data layer tests

09

Authentication & Security

9. Authentication & Security

Authentication Paths

Browser User → Microsoft Entra ID (OIDC)
Microsoft.Identity.Web
JWT Bearer tokens

API Consumer → API Key header (X-API-Key)
AuthorizeApiKey attribute
Falls back to anonymous

Azure Services → DefaultAzureCredential
Managed Identity (User + System)
No passwords in config

Three auth paths: Browser (OIDC), API consumer (API Key), Azure services (Managed Identity)

10

Theme and UI

10. Theme and UI

MudBlazor 9.2.0 component library throughout

Light/dark theme toggle stored in browser localStorage

Retro 90s theme easter egg (toggled via theme key "nineties")

Scoped CSS per component via .razor.css pattern

Bootstrap spacing utilities and CSS variables

Responsive layout — tested across viewport sizes

11

Configuration Reference

11. Configuration Reference

Setting	Description
DataSource	JSON or SQL — selects data repository
DefaultConnection	SQL connection string (when DataSource = SQL)
KeyVaultName	Azure Key Vault name for secret loading
APPLICATIONINSIGHTS_CONNECTION_STRING	App Insights telemetry endpoint
AzureOpenAIChatEndpoint	Azure OpenAI chat endpoint URL
AzureOpenAIChatDeploymentName	GPT-4o deployment name
AzureOpenAIChatApiKey	OpenAI API key (or use Managed Identity)
AzureOpenAllImageEndpoint	DALL-E endpoint URL
AzureOpenAllImageDeploymentName	DALL-E 3 deployment name
AzureOpenAllImageApiKey	Image API key

12

Key Design Patterns

12. Key Design Patterns

Pattern	Where Used	Description
Repository Pattern	DadABase.Data	IJokeRepository abstracts data source (JSON vs SQL)
Code-Behind	All Blazor pages	.razor + .razor.cs separation of concerns
Scoped CSS	All components	.razor.css per component
Dependency Injection	Program.cs	All services registered via DI
Options Pattern	AppSettings	Typed configuration object
Dual Data Source	Data layer	Runtime switch between JSON and SQL
API Key Auth	REST controllers	Custom attribute-based gate
Managed Identity	Azure connections	DefaultAzureCredential throughout
Agent Framework	AIHelper.cs	Multiple AIAgent instances for different tasks
DACPAC	Database project	Schema-as-code with automated deployment

13

Copilot / GitHub Copilot Configuration

13. Copilot / GitHub Copilot Configuration

GitHub Copilot configuration hierarchy

Path	Purpose
copilot-instructions.md	Global coding conventions for Copilot
instructions/	Domain-specific instruction files (7 files)
skills/	Specialized Copilot custom skills (100+ skills)
agents/	Custom Copilot agent definitions (34 agents)
prompts/	Reusable prompt templates (12 files)
actions/	Reusable composite GitHub Actions (load-project-config/, login-action/)
config/	Repository-level configuration files
scripts/	Utility scripts

14

Azure Developer CLI (AZD) Integration

14. Azure Developer CLI (AZD) Integration

AZD integration files

File	Purpose
azure.yaml	AZD service and infrastructure mapping
infra/azd-main.bicep	AZD-specific Bicep entry point
infra/azd-main.parameters.json	AZD parameter defaults
.github/workflows/azure-dev.yml	AZD CI/CD integration

15

Documentation

15. Documentation

Project documentation index

Document	Location	Description
README	README.md	Project overview and quick start
Coding Standards	Docs/Coding_Standards.md	Team conventions
Infrastructure as Code	Docs/Infra_As_Code.md	Bicep deployment guide
SQL DACPAC	Docs/SQL-DacPac.md	Database deployment guide
Azure DevOps Pipelines	Docs/YML_AzDO.md	AzDO pipeline reference
GitHub Actions	Docs/YML_GitHub.md	GHA workflow reference
Deployment Options	Docs/Deployment_Options.md	Deployment method comparison
Deployment Quick Ref	Docs/Deployment_QuickRef.md	Quick reference card
Export Functionality	Docs/ExportFunctionality.md	Data export guide
SQL Permissions	Docs/SQL-Permissions-Queries.md	SQL permission reference

Dad-A-Base

Architecture Documentation